# Unit Test Report

**Project:** Voice Assistant for Multimedia Controls - Team 3 **Prepared for:** Dhanush S S, Integration Lead **Execution date:** 07 November 2026, 01:50 IST (UTC+05:30) **Application source commit:** `91d5a74d815a2daff04afd896e4f135877d552b9` **Test mode:** author-run automated host tests; independent acceptance pending.

The course testing window shown in the brief is **2-6 November**. These tests were actually run on **7 November 2026 IST**; results are not backdated. UTC timestamps may fall on 6 November. Report date and evidence date are distinct from the course deadline.

## Result

**PASS for the executed host suites:** PlatformIO native **93/93**, independently rebuilt instrumentation run **93/93**, and optional Python prototype **10/10**. The instrumentation run repeats the same 93 existing cases; it is not 93 new tests. Distinct automated test cases/methods reported: **103** (93 C++ + 10 Python). Subtests within Python methods are not counted as extra methods.

The C++ suite contains **87 unit/contract cases and 6 downstream integration cases**. The integration cases are reported separately from unit acceptance.

## Executed suites

| Existing suite | Test methods/cases | Failures | Classification |
|---|---|---|---|
| test_DR_01_contracts | 3 | 0 | Host unit/contract |
| test_INT_01 | 6 | 0 | Host integration |
| test_VAMC_F_005 | 12 | 0 | Host unit/contract |
| test_VAMC_F_006 | 12 | 0 | Host unit/contract |
| test_VAMC_F_007 | 5 | 0 | Host unit/contract |
| test_VAMC_F_008 | 6 | 0 | Host unit/contract |
| test_VAMC_F_009 | 4 | 0 | Host unit/contract |
| test_VAMC_F_010 | 6 | 0 | Host unit/contract |
| test_VAMC_F_011 | 9 | 0 | Host unit/contract |
| test_VAMC_F_012 | 5 | 0 | Host unit/contract |
| test_VAMC_F_013 | 3 | 0 | Host unit/contract |
| test_VAMC_F_014 | 5 | 0 | Host unit/contract |
| test_VAMC_SR_002 | 5 | 0 | Host unit/contract |
| test_VAMC_SR_003 | 6 | 0 | Host unit/contract |
| test_reset_button | 6 | 0 | Host unit/contract |

Python: `companion/laptop_prototype/tests/test_prototype.py` - 10 methods, 0 failures. It uses mocks/fakes for audio, transcription and media dispatch; actual Windows key injection, serial recording and Whisper inference were not run.

## What was exercised

• Seven-command text matching/normalization, phrase variants, ambiguity and rejection.

• HID mapping, send policy, disconnected link, bounded retry/backoff and feedback.

• Provisioning start/input validation, copied credentials, timeout and protected-save errors.

• Portable discovery/passkey policy, reset coordinator and five-second BOOT hold detector.

• Circular-log behavior and foundation contracts.

• Real native OpenSSL credential round trips, wrong passphrase, tampering, fresh nonce, erase, malformed records and storage failures. ESP32 mbedTLS/NVS execution is untested.

• Downstream gate > mapping > fake BLE transport > feedback integration, covering all seven actions, rejection, recognition failure, disconnect, retry and exhaustion.

• Python classifier/mapper, dry-run CLI, rejection, platform guard and cleanup under injected transcription failure.

## Missing tests and limits

No on-board wake-word engine, microphone capture, tinyML inference, real authenticated BLE/HID behavior, GPIO output, actual flash inspection, OTA or battery test ran. An empty/fake adapter and a passing text test cannot establish those features. The actual target PlatformIO build failed due to missing configuration; see the build output. No ESP32 serial port was detected.

No coverage threshold was added and no tests were altered to inflate results. Cross-testers must independently run the linked STP cases and record A/F themselves.

## Scope and acceptance boundary

The approved project uses ESP32-WROOM and offline fixed-vocabulary recognition. No live cloud API, microphone, flash operation or Bluetooth/media action was performed by this report run. Passing host tests is not completion of the physical SRS/STP acceptance criteria. This report does not set RTM A/F statuses, close issues, complete INT-02, create a release tag or freeze development.

The locally revised SRS/SAD/STP/backlog implement Dhanush's offline redesign; some revisions are still unpublished and remote source text remains inconsistent. The exact source-document hashes are in source-baselines.json. Harsh/DOC-01 must reconcile and publish those baselines before final acceptance. Legacy `TC-ASR-*` IDs are retained; their names do not authorize cloud recognition.

## Reproduction and evidence

From the repository root, with PlatformIO installed:

```
pio test -d firmware -e native
python3 -m pip install coverage==7.17.2
python3 scripts/measure_coverage.py --output /tmp/vamc-report-rerun
```

The coverage runner currently supports **macOS** with Xcode command-line tools, Homebrew `openssl@3` and Unity installed by the PlatformIO test run. It compiles the same existing C++ tests and library sources with Apple Clang/LLVM 21 at `-O0`, using `-fprofile-instr-generate -fcoverage-mapping`. Test/Unity code is not instrumented. All 12

portable `.cpp` objects are linked into every suite; the 15 raw profiles are merged once, and one binary provides their shared mappings. This avoids counting shared code 15 times. The runner then invokes Python coverage with `--branch`, including all six prototype source files and excluding its tests. It records tool versions, commands, source commit and suite counts.

- Run manifest and exact commands
- Coverage scope inventory
- PlatformIO native test output
- C++ coverage summary
- C++ annotated lines and branch counts
- C++ machine-readable coverage
- Python test output
- Python coverage summary
- Python machine-readable coverage
- Python annotated HTML

Method reference: Clang source-based coverage documentation. Execution evidence, rather than documentation examples, supplies every result here.